

Question 1:

We divide to two n meters steel, for example 8 meters steel is 4, 4. 7 meters steel is 4, 3. After that we divide regular steels to two again, after one under the other.

Analyze:

$$T(n) = \begin{cases} T(n) = 1 \rightarrow \text{best case} \\ T(n) = T(n/2) + 1 \rightarrow \text{worst case} \end{cases}$$

Let's using master theorem

$$a=1, b=2, f(n)=1 \Rightarrow n^{\log_2 1} \stackrel{?}{=} 1 \Rightarrow n^0 \stackrel{V}{=} 1 \quad n^{\log_2 a} \in O(f(n))$$

$$\text{So, } T(n) = O(n^{\log_2 a} \cdot \log n) \Rightarrow \boxed{T(n) = O(\log n)}$$

Question 2: if we sort the experiment results using divide and conquer algorithm (we used quick sort), the first element will be worst result and the last element will be best result

Analyze:

Best Case: The best case occurs when the partition process always picks the middle element as pivot.

$$T(n) = 2T(n/2) + n$$

Let's using master theorem

$$a=2, b=2, f(n)=n \Rightarrow n^{\log_2 2} \stackrel{?}{=} n \Rightarrow n^1 \stackrel{V}{=} n \quad n^{\log_2 a} \in O(f(n))$$

$$\text{So, best case is } T(n) = O(n \cdot \log n)$$

Worst Case: The worst case occurs when the partition process always picks greatest or smallest elements as pivot.

$$T(n) = T(n-1) + n, \quad T(1) = 0$$

Let's apply backward substitution:

$$T(n) = n + T(n-1)$$

$$T(n-1) = n-1 + T(n-2)$$

$$T(n-2) = n-2 + T(n-3)$$

$$T(3) = 3 + T(2)$$

$$T(2) = 2 + T(1)$$

$$T(1) = 0$$

$$T(n) = n + (n-1) + (n-2) + \dots + 3 + 2$$

$$= \frac{n^2}{2} \Rightarrow T(n) = O(n^2)$$

Let's prove by induction prove:

1) $T(n) = T(n-1) + n$ if $n=2$ answer is 2

$T(n) = \frac{n^2}{2}$ if $n=2$ answer is 2 ✓

2) Assume $T(n-1) = \frac{(n-1)^2}{2}$

3) $T(n) = T(n-1) + n = \frac{(n-1)^2}{2} + n = \frac{n^2}{2}$ ✓

Question 3: The idea is not to complete Quicksort, but stop at the point where pivot itself is k -th smallest element

Analyse: The worst case corresponds to the longest possible execution of the algorithm. It's the one in which the k -th largest element is the last one standing, and there are n recursive calls.

$$T(n) = T(n-1) + n \Rightarrow T(n) = O(n^2).$$

This analyse was done Question 2.

Question 4: The idea is similar to mergesort. Divide the array into two equal or almost equal halves in each step until the base case is reached. Create a function merge that counts the number of reverse order pairs. When two halves of the array are merged, create two indices i and j , i is index of the first half and j second half. If $a[i] > a[j]$ then there are $(mid - i)$ reverse order. Because left and right subarray are sorted. So all the remaining elements in left sub array will be greater than $a[j]$.

Analyze: The algorithm uses divide and conquer.

$$T(n) = 2T(n/2) + n$$

Let's using master method:

$$a=2, b=2, f(n)=n \Rightarrow \begin{matrix} n^{\log_2 2} = n \\ n^{\log_2 a} = f(n) \end{matrix} \Rightarrow n^1 = n \quad n^{\log_2 a} \in O(f(n))$$

$$\text{So, } T(n) = O(n^{\log_2 a} \cdot \log n) \Rightarrow T(n) = O(n \cdot \log n)$$

Question 5:

Brute force: The idea is multiply number with itself base number times. ($3^4 = 3 \cdot 3 \cdot 3 \cdot 3$)

Analyze:

$$\sum_{i=1}^n 1 = n = T(n) = O(n)$$

Base number $n \rightarrow$ multiply operation $O(n)$

Divide and Conquer: In divide and conquer approach, exponential of x^n is achieved by creating sub problems of size $x^{n/2}$. They are subproblems are divided recursively until they hit to the power 1.

Analyze:

$$T(n) = T(n/2) + 1$$

Let's using master theorem:

$$a=1, b=2, f(n)=1 \Rightarrow \begin{matrix} n^{\log_2 1} = 1 \\ n^{\log_2 a} = f(n) \end{matrix} \Rightarrow n^0 = 1 \quad n^{\log_2 a} \in O(f(n))$$

$$\text{So, } T(n) = O(n^{\log_2 a} \cdot \log n) \Rightarrow T(n) = O(\log n)$$